

Mesh Curving and Refinement Based on Cubic Bézier Surface for High-Order Discontinuous Galerkin Methods

Shu-Jie Li

Beijing Computational Science Research Center (CSRC), Building 9 Zhongguanchun Park II, Beijing, 100193 China

e-mail: shujie@csrc.ac.cn

Received June 6, 2019; revised June 6, 2019; accepted August 5, 2019

Abstract—In this paper, three-dimensional mesh curving and refinement methods are examined for high-order flow simulations with discontinuous Galerkin (DG) methods on hybrid grids. The mesh curving algorithm converts linear surface elements to quadratic ones with the cubic Bézier surface reconstruction. The effects of mesh curving on the impacts of DG solutions of the Euler and Navier–Stokes equations are investigated. Numerical results show that significant enhancements of accuracy and robustness can be gained for DG solutions of smooth and discontinuous flow fields. Additionally, a curved mesh refinement algorithm is also realized by inquiring the midpoints of edges and faces of the reconstructed quadratic elements. With this method, up to 0.9 billions curved elements are successfully generated around the DLR-F6 wing/body/nacelle/pylon configuration.

Keywords: mesh curving, mesh refinement, discontinuous Galerkin method

DOI: 10.1134/S0965542519120133

1. INTRODUCTION

High-order methods of computational fluid dynamics (CFD) are proven useful in many aspects, e.g., the discontinuous Galerkin methods (DG) [1–8]. However, the total numerical error of using a high-order method can be affected by the approximation error of surface geometries, e.g., see reference [7], in which using curved elements are shown to be necessary to eliminate the unphysical entropy wake in DG solvers. As a consequence, using curved elements has been widely accepted by the high-order CFD community and considering the curvature of surface elements becomes the minimum requirement of achieving high-fidelity simulations, as demonstrated by Lilia [8]. However, for practical applications, such a requirement is not always met. Usually, only straight-sided meshes are available for readily use while the CAD files are inaccessible, which prevents practical applications of high-order flow solvers requiring curved grids. To solve this issue, a few efforts have been made, of which the generated curved elements are used for their own flow solvers with spectral element or CRP/FR methods [19–22]. Among which, Gao et al. [20] developed a mesh curving method with the Bézier curves for 2D Euler equation. In this paper, we extend the Bézier curves reconstruction to 3D hybrid grids based the cubic Bézier surfaces reconstruction [12, 13], which converts linear elements to quadratic elements. Furthermore, a useful function is also realized to refine curved meshes for high-order DG computations with massively curved elements. And, for the first time, up to 0.9 billions curved elements are successfully generated around the DLR-F6 wing/body/nacelle/pylon configuration [14], demonstrating a potential use for very large-scale industry applications of high-order methods.

The remainder of this paper is organized as follows. Section 2 discusses the mesh curving method. Section 3 introduces spatial discretizations which include a discontinuous Galerkin discretization of the Navier–Stokes equations and the integration of curved surfaces. Additionally, a curved mesh refinement method is also presented by inquiring the midpoints of edges and faces of the reconstructed quadratic elements. Section 4 investigates the impacts of applying the reconstructed curved elements to DG solutions of inviscid and viscous flows. The results of massively curved elements to the DLR-F6 wing/body/nacelle/pylon configuration are also given. Finally, Section 5 concludes this paper.

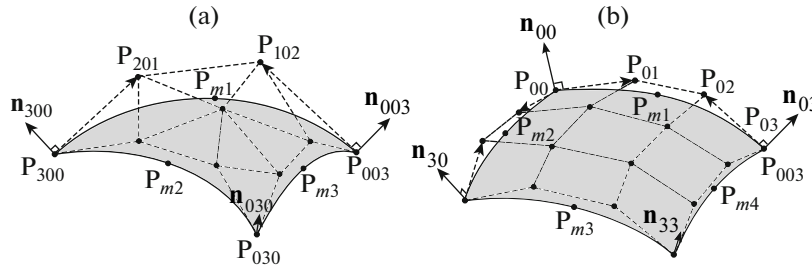


Fig. 1. Cubic Bézier surfaces: triangle (a), quadrilateral (b).

2. MESH CURVING

2.1. Cubic Bézier Surface

The cubic Bézier surface reconstruction [12, 13] is used for mesh curving of 3D hybrid grids. For most cases, there exists two types of surface elements, namely, triangles and quadrilaterals. Therefore, the cubic Bézier triangle and quadrilateral reconstructions are used which are illustrated in Figs. 1a and 1b.

To construct quadratic curved elements compatible to general quadratic isoparametric finite element methods, extra mid-edge points (such as p_{m1} , p_{m2} , p_{m3} in Figs. 1a and 1b) need to be computed with the cubic Bézier curves which requires the cubic Bézier reconstruction in advance. Not loss of generality, we consider a n -order of Bézier curve in the form of

$$\mathbf{B}(t) = \sum_{i=0}^n \mathbf{b}_{i,n}(t) \mathbf{P}_i, \quad t \in [0, 1], \quad (1)$$

where $\mathbf{P}_i$ is the i_{th} control node of the Bézier curve, $\mathbf{B}(t)$ is the resulting parametric curve and term $\mathbf{b}_{i,n}(t)$ is called the *Bernstein basis polynomials* which may be written as

$$\mathbf{b}_{i,n}(t) = \binom{n}{i} t^i (1-t)^{(n-i)}, \quad i = 1, \dots, n. \quad (2)$$

Note that $t^0 = 1$, $(1-t)^0 = 1$ for any t value, and that the *binomial coefficient* $\binom{n}{k}$ is defined as

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}. \quad (3)$$

The first and last control nodes coincide with the end points of the edge, so only the two internal control nodes are computed. The derivative of the Bézier curve with a respect to t is actually a tangential vector $\mathbf{T}$:

$$\mathbf{T}(t) = \mathbf{B}'(t) = n \sum_{i=0}^{n-1} (\mathbf{P}_{i+1} - \mathbf{P}_i) \cdot \mathbf{b}_{i,n-1}(t). \quad (4)$$

The cubic Bézier curve with $n = 3$ is shown in Fig. 1a. This triangle consists of three corner nodes $\mathbf{P}_{300}$, $\mathbf{P}_{030}$, $\mathbf{P}_{003}$ and three computed surface normals $\mathbf{n}_{300}$, $\mathbf{n}_{030}$, $\mathbf{n}_{003}$. To reconstruct the cubic Bézier curve for each edge of the triangle, for instance on the edge $\mathbf{P}_{003} \rightarrow \mathbf{P}_{300}$, the tangential vector is computed as Eq. (4) where $\mathbf{T}(0)$ and $\mathbf{T}(1)$ must be in a same vector direction.

$$\begin{aligned} \mathbf{T}(0) &= \mathbf{P}_{300} - \mathbf{P}_{003} - ((\mathbf{P}_{300} - \mathbf{P}_{003}) \cdot \mathbf{n}_{030}) \mathbf{n}_{030}, \\ \mathbf{T}(1) &= \mathbf{P}_{300} - \mathbf{P}_{003} - ((\mathbf{P}_{300} - \mathbf{P}_{003}) \cdot \mathbf{n}_{300}) \mathbf{n}_{300}. \end{aligned} \quad (5)$$

By solving Eq. (5), the control nodes are expressed as

$$\begin{aligned} \mathbf{P}_0 &= \mathbf{P}_{003}, \\ \mathbf{P}_1 &= \mathbf{P}_{102} = (2\mathbf{P}_{003} + \mathbf{P}_{300} - ((\mathbf{P}_{300} - \mathbf{P}_{003}) \cdot \mathbf{n}_{030}) \mathbf{n}_{030})/3, \\ \mathbf{P}_2 &= \mathbf{P}_{201} = (2\mathbf{P}_{300} + \mathbf{P}_{003} + ((\mathbf{P}_{300} - \mathbf{P}_{003}) \cdot \mathbf{n}_{300}) \mathbf{n}_{300})/3, \\ \mathbf{P}_3 &= \mathbf{P}_{300}. \end{aligned} \quad (6)$$

With the cubic Bézier curve formula Eq. (1) defined by Eq. (6), the middle node of each edge can be computed by taking $t = 1/2$. For both six-node triangle and eight-node quadrilateral, not mid-face nodes exist, thus the reconstruction is actually done on edges but not surfaces. The same procedure can be applied to all the edges on the wall surfaces. Finally, the curved elements can be output into a mesh file for high-order flow solvers.

2.1.1. Reconstruction of normals. A cubic Bézier surface is defined by nodal coordinates and normals, as nodal coordinates are already supplied by the linear mesh only normals need to be computed. A simple, yet effective choice of approximating normals is the inverse-distance weighting (IDW) method [15], which approximates the normal $\mathbf{n}_i$ at the i th node with its N surrounding surface normals as in Eq. (7). And r_j is the distance from the i th node to its j th neighboring surface center.

$$\omega_i = \frac{r_i^{-2}}{\sum_{j=1}^N r_j^{-2}}, \quad \mathbf{n}_i = \sum_{j=1}^N \omega_j \mathbf{n}_j. \quad (7)$$

The IDW method satisfies $\sum_{j=1}^N \omega_j = 1$ and it is thus linear preserving. And, it also produces nonlinear interpolation as data vectors distribute nonlinearly. One of the greatest advantages of using IDW is that the interpolated normals can be bounded by the maximum and minimum of the data set, thus prevents introducing any dangerous new extremum. Therefore, the safety of normal reconstruction is enhanced. Other types of nonlinear normal approximations are also applicable but must avoid introducing any new extremum.

The normals are then computed for all the wall nodes which are the most significant parts causing geometric errors. For each node, a priority of patch ordering is predefined to make sure one node belongs to one particular patch for avoiding node singularity. The patch ordering is defined as follows: (1) solid wall surfaces, (2) symmetric wall surfaces (3) other else. At each node, a single normal is computed despite that its surrounding surface cells could be on different patches if no patch label is defined in the provided linear mesh. Under this circumstance, a node could belong to the intersection of multiple patches, the computed normal might have vector directions different greatly to their surrounding normals. To detect such singularities, the angle α between two end-point normals of an edge is used. For which the edges with $\alpha \geq 30^\circ \sim 60^\circ$ are kept straight. The remaining edges are proceed with the cubic Bézier reconstruction. For volume elements, mid-edge nodes are obtained by geometric averages of end-point nodes. Numerical tests of the current mesh curving strategies for 3D geometries are given in Section 4, and the effectiveness of mesh curving in DG solutions of 3D flows are investigated.

3. SPATIAL DISCRETIZATIONS

In this section, spatial discretizations on the curved elements framework are described. A discontinuous Galerkin flow solver HA3D with the physical orthogonal basis [1–6] is used for discretizing the compressible Navier–Stokes equations. Quadratic isoparametric elements are used for the surface and volume integrations which are derived from the cubic Bézier reconstruction. A mesh refinement method is further developed as an enhanced function for future large-scale computations with massively curved elements.

3.1. Discontinuous Galerkin Method

Consider the three-dimensional compressible Navier–Stokes equations

$$\mathbf{U}_t + \nabla \cdot \mathbf{F}^c(\mathbf{U}) + \nabla \cdot \mathbf{F}^v(\mathbf{U}, \nabla \mathbf{U}) = \mathbf{S}(\mathbf{U}), \quad (8)$$

where $\mathbf{U}, \mathbf{S}$ denote the conservative vector and source term, $\mathbf{F}, \mathbf{F}^{vis}$ are the inviscid and viscous flux, respectively.

$$\mathbf{U} = \begin{pmatrix} \rho \\ \rho \mathbf{v} \\ \rho E \end{pmatrix}, \quad \mathbf{F}^c = \begin{pmatrix} \rho(\mathbf{v} - \mathbf{v}_r)^T \\ \rho(\mathbf{v} - \mathbf{v}_r)\mathbf{v}^T + \mathbf{p}\mathbf{I} \\ \rho \mathbf{H}(\mathbf{v} - \mathbf{v}_r)^T \end{pmatrix}, \quad \mathbf{F}^v = \begin{pmatrix} 0 \\ -\tau \\ \mathbf{q} - \tau \mathbf{v} \end{pmatrix}, \quad \mathbf{S} = \begin{pmatrix} 0 \\ -\rho \boldsymbol{\omega} \times \mathbf{v} \\ 0 \end{pmatrix}, \quad (9)$$

where $\mathbf{v} = (u, v, w)^T$ is the absolute velocity, $\boldsymbol{\omega} = (\omega_x, \omega_y, \omega_z)^T$ is the angular velocity of the rotating frame of reference, $\mathbf{v}_r = \boldsymbol{\omega} \times \mathbf{x}$; $\boldsymbol{\tau}$, $\mathbf{q}$ the viscous stress bf and the heat flux vector $\mathbf{p}$, p , and e denote the flow density, pressure, and the specific internal energy; $E = e + \frac{1}{2}\|\mathbf{v}\|^2$ and $H = E + p/\rho$ denote the total energy and total enthalpy, respectively; $\mathbf{I}$ denotes the 3×3 unit matrix; and the pressure p is given by the equation of state for a perfect gas

$$p = \rho(\gamma - 1)e, \quad (10)$$

where $\gamma = 7/5$ is the ratio of specific heats for perfect gas.

The discontinuous Galerkin discretization of Eq. (8) is defined on a computational domain Ω divided into a set of non-overlapping elements of arbitrary shape. The modal discontinuous Galerkin method seeks an approximation $\mathbf{U}_h$ in each element $E \in \Omega$ with finite dimensional space of polynomial $\{P_p(E) \in L_2(\Omega), \forall E \in \Omega\}$ of order p , namely

$$\mathbf{U}_h(\mathbf{x}, t) = \sum_{j=1}^n \mathbf{u}_j(t) \psi_j(\mathbf{x}). \quad (11)$$

The weak form of Eq. (8) reads

$$\int_E \psi_i \psi_j d\mathbf{x} \frac{d\mathbf{u}_j}{dt} = - \int_{\partial E} \psi_i \tilde{\mathbf{F}} \cdot \hat{\mathbf{n}} d\sigma + \int_E (\mathbf{F} \cdot \nabla \psi_i + \psi_i \mathbf{S}) d\mathbf{x} := \mathbf{R}_i, \quad (12)$$

where the Einstein summation convention is used. Here $\hat{\mathbf{n}}$ denotes the outwards unit normal of the surface element ∂E of the element E . The flux terms are defined as

$$\tilde{\mathbf{F}} = \tilde{\mathbf{F}}^c(u_h^\pm) + \tilde{\mathbf{F}}^v(u_h^\pm, (\nabla_h u_h + \boldsymbol{\delta}_f)^\pm); \quad \mathbf{F} = \mathbf{F}^c(u_h) + \mathbf{F}^v(u_h, (\nabla_h u_h + \boldsymbol{\delta})), \quad (13)$$

where $\tilde{\mathbf{F}}^c$ is computed with the HLLC flux [9], and the viscous flux $\tilde{\mathbf{F}}^v$ is computed with the second approach of Bassi and Rebay (BR2) which introduces the local and global lifting operators $\boldsymbol{\delta}_f$ and $\boldsymbol{\delta}$ [11]. The BR2 viscous flux discretization is compact where the global lifting operator is used for volume integration while the local lifting operator is used face-wised

$$\boldsymbol{\delta} = \sum_{\partial E} \boldsymbol{\delta}_f, \quad (14)$$

and the local lifting operator is solved by Galerkin projection on each surface $\boldsymbol{\sigma}$, where we introduce the average operator $\{a\} = \frac{1}{2}(a^+ + a^-)$

$$\int_{\partial E} \boldsymbol{\psi} \cdot \boldsymbol{\delta}_f = \int_{\partial E} \boldsymbol{\psi} \cdot (\{u\} - u_h^+) d\boldsymbol{\sigma}. \quad (15)$$

The BR2 formulation overcomes the problem of sub-optimal convergence rates occurred in the BR1 method.

3.2. Curved Surface Integration

The surface integral $\tilde{\mathbf{F}}$ of Eq. (12) has to be discretized with considering of curved geometries. For curved surface elements, the standard linear surface is replaced with a polynomial representation on a reference element $\hat{\Omega}$

$$\mathbf{x} = \sum_{k=1}^{np} x_k N_k(\boldsymbol{\xi}), \quad \forall \boldsymbol{\xi} \in \hat{\Omega}. \quad (16)$$

The transformation from the reference element to the physical curved element is expressed in terms of Lagrangian basis N_k and surface mapping Jacobian J_s . Therefore, the surface integral $\tilde{\mathbf{F}}$ is approximated by the Gaussian Legendre law of ng quadrature points according to the order requirement of DG.

$$\int_{\partial E} \psi_i \tilde{\mathbf{F}} \cdot \hat{\mathbf{n}} d\sigma \approx \sum_{ig=1}^{ng} \omega_{ig} \psi_i(x(\boldsymbol{\xi}_{ig})) \tilde{\mathbf{F}}(x(\boldsymbol{\xi}_{ig})) J_s(\boldsymbol{\xi}_{ig}). \quad (17)$$

Table 1. Quadratic surface elements

Shape function	Six-node triangle	Eight-node quadrilateral
N_1	$\xi(2\xi - 1)$	$\frac{1}{4}(1 - \xi)(\eta - 1)(\xi + \eta + 1)$
N_2	$\eta(2\eta - 1)$	$\frac{1}{4}(1 + \xi)(\eta - 1)(\eta - \xi + 1)$
N_3	$\zeta(2\zeta - 1)$	$\frac{1}{4}(1 + \xi)(\eta + 1)(\xi + \eta + 1)$
N_4	$4\xi\eta$	$\frac{1}{4}(\xi - 1)(\eta + 1)(\xi - \eta + 1)$
N_5	$4\eta\zeta$	$\frac{1}{2}(1 - \eta)(1 - \xi^2)$
N_6	$4\xi\zeta$	$\frac{1}{2}(1 - \xi)(1 - \eta^2)$
N_7	n/a	$\frac{1}{2}(1 + \eta)(1 - \xi^2)$
N_8	n/a	$\frac{1}{2}(1 - \xi)(1 - \eta^2)$

The resulting quadratic elements approximate the curvature of cubic Bézier surfaces but are immediately compatible with traditional finite element solvers. The mapping from reference coordinate $(\xi, \eta) \in [(0, 1), (0, 1)]$ to physical coordinate $\mathbf{x}$ on a triangle is defined as $\mathbf{x} = \sum_{i=1}^6 N_i \mathbf{x}_i$ where $\zeta = 1 - \xi - \eta$.

For a quadrilateral, the reference coordinate is $(\xi, \eta) \in [(-1, 1), (-1, 1)]$ and $\mathbf{x} = \sum_{i=1}^8 N_i \mathbf{x}_i$. The midpoints of edges can be obtained setting $t = 1/2$ with Eq. (1) of the Bézier curve. These shape functions are listed in Table 1. The volume element integration are also integrated with quadratic elements, which are similar to the treatment of surface integrals. Note that the elements adjacent to the wall surfaces are truly curved and the elements inside the domain are not necessarily curved for most cases. For those cases where volume curving are important, advanced global mesh optimizations are required however beyond the scope of this paper. We then consider a curved mesh refinement method based on these quadratic elements base on which any state-of-art mesh optimization method can be applied additionally.

3.3. Curved Mesh Refinement

In this section, a curved mesh refinement tool is presented based on the newly reconstructed quadratic elements in Section 2. Global mesh refinements are realized by inquiring curved mesh information such as the midpoints of edges and faces. To avoid duplicating the node storage, edge- and face-based arrays are created for storing newly generated middle nodes of edges and faces. A convention of elemental node numbering has to be predefined to form subcell topologies and must keep consistent during mesh refinements.

The mesh refinement algorithm is detailed in Algorithm 1. We explain several key steps below. First, the refinement process is iteratively called with storing an internal mesh file for each level of mesh refinement. Note that only the initialization stage requires the participation of cubic Bézier curves and the refinement levels greater than two use the shape function interpolations instead for saving computational cost. The resulting quadratic elements approximate the curvature of cubic Bézier surfaces but have the advantage that are immediately compatible with traditional finite element solvers.

In the step 4, the mesh curving information is actually stored in the quadratic elements. The midpoints of the quadratic elements are computed using Bézier curves in the initial step and then switch to use the quadratic elements provided by the last level of mesh. In the step 11, according to paper [18], there are three ways of conducting tetrahedron splitting. Among all the splitting possibilities, we split the tetrahedron with choosing the shortest average edge length of the new children cells. In the step 13 of Algorithm 1, the

quadratic children cells are computed by computing the mid-edge points using the shape functions of Table 1. The mesh refinement algorithm is applied to the DLR-F6 configuration which will be given in the next section.

Algorithm 1 Curved mesh refinement

```

1: Recover cubic Bézier surfaces from the provided linear mesh
2: for Each level of mesh refinement  $iLevel$  do
3:   if  $iLevel = 1$  then
4:     Compute mid-edge & mid-face nodes from the cubic Bézier surfaces
5:   else
6:     Read in internal mesh file
7:   end if
8:   Form quadratic isoparametric elements
9:   Update face list & edge list
10:  if find tetrahedron then
11:    Split tetrahedron with considerations of mesh quality [18]
12:  end if
13:  Split quadratic isoparametric elements and store quadratic children cells
14:  Dump the internal mesh file
15: end for

```

4. NUMERICAL RESULTS

In this section, numerical tests of the mesh curving strategy is investigated in four 3D cases including Taylor Couette flow, flow past a sphere, ONEAR M6 wing and DLR-F6 wing/body/nacelle/pylon configuration. In the Taylor Couette flow, the implementation of discontinuous Galerkin method of the Navier–Stokes equations is verified on quadratic curved elements. In the sphere flow and ONERA M6 wing cases, numerical errors are examined in terms of theory and experiment. We then consider the DLR-F6 wing/body/nacelle/pylon configuration for assessing the basic capability of mesh curving around complex geometries as well as generating massively curved elements through curved mesh refinements. Finally, up to 0.9 billions curved cells are generated as an illustrate case for next-generation, large-eddy simulations (LES) of turbulent flows.

4.1. Taylor Couette Flow

The implementation of high-order discontinuous Galerkin discretization for the Navier–Stokes equations on quadratic curved elements is verified with the Taylor Couette flow. The fluid is driven by two concentric cylinders which are rotating with constant angular velocities of ω_0 and ω_1 . A low Reynolds number $Re = 10$ for maintaining a laminar state is used which is defined by the tangential velocity and the radius of the inner cylinder. Constant viscosity is used to allow comparisons with the analytical solution, which is given below

$$u_\theta = \omega_0 r_0 \frac{r_1/r - r/r_1}{r_1/r_0 - r_0/r_1} + \omega_1 r_1 \frac{r/r_0 - r_0/r}{r_1/r_0 - r_0/r_1}, \quad (18)$$

where u_θ is the tangential velocity, $r_0 = 1$ and $r_1 = 2$ are the inner radius and the outer radius. The isothermal boundary condition is set on the inner cylinder with angular velocity ω_0 at Mach number $M_0 = 0.2$. The outer cylinder is stationary with $\omega_1 = 0$ and the adiabatic wall boundary condition is used. The front and back faces in the z -direction use the symmetric boundary condition for quasi-2D computations. The order of accuracy is computed on a sequence of three meshes, using first- to fifth-order DG schemes ($P^{0 \rightarrow 4}$). Quadratic curved elements are used on the boundary surfaces. Figure 2a shows the coarsest mesh of the refinement sequence. The expected velocity contours are also displayed in Fig. 2b. Finally, the

L_2 norms of velocity errors are detailed in Table 2. They are computed as $\left[\int (u - u_\theta)^2 d\Omega / \int d\Omega \right]^{1/2}$ integrating in the entire computational domain Ω . The expected order of convergence is observed for all the p lev-

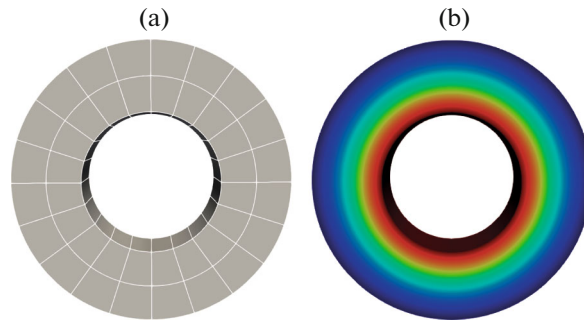


Fig. 2. Taylor Couette flow: (a) the coarsest mesh 20×2 , (b) velocity magnitude contour of the P_4 solution.

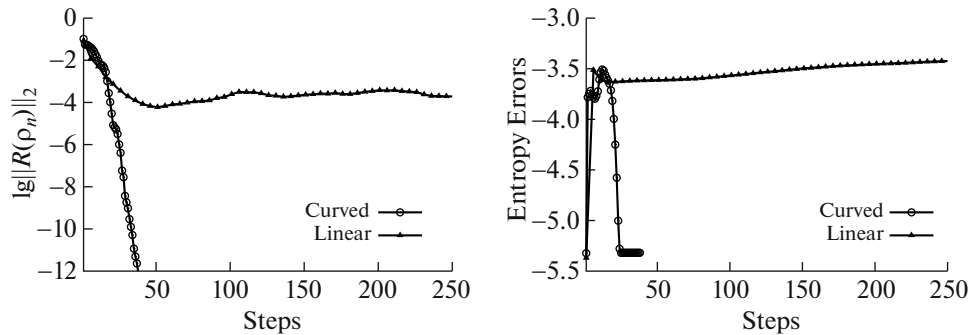


Fig. 3. Smooth flow past a sphere: convergence history of the density residuals $\log \|R(p_n)\|_2$ with the curved mesh (left) and the linear mesh (right). Rapid convergence can be achieved by using the curved mesh while the unphysical wake flow with the linear mesh leads to residual stagnation.

els, thereby verifying the high-order implementation of DG viscous discretization and also the use of curved elements.

4.2. Flow Past a Sphere

In this case, a smooth flow around a sphere is computed at Mach number $M_\infty = 0.3$. Symmetric flow patterns and the constant entropy are admitted in theory so that numerical error due to curved mesh approximations can be measured quantitatively.

The fourth-order (P_3) DG spatial discretization and a fast first-order exponential time integration (EXP1) method [4] with $CFL = 10^3$ is used for the steady-state time marching. Both linear and curved meshes are tested, which show that the unphysical wake flow produced by using the linear elements is greatly improved by using the curved mesh, see also [7, 8]. Figure 3 presents the steady-state residual convergences of the linear and curved meshes. It is so obvious that meaningful high-order accurate solutions

Table 2. Laminar Taylor Couette flow: validation of the order of accuracy with the analytical solution. The L_2 error is expressed in the $\log_{10}$ scale

Mesh	20×2	40×4	80×8	Order
P_0	-7.962E-01	-1.130E+00	-1.435E+00	1.05
P_1	-2.473E+00	-2.378E+00	-3.459E+00	1.68
P_2	-2.473E+00	-3.459E+00	-4.418E+00	3.23
P_3	-3.293E+00	-4.542E+00	-5.752E+00	4.08
P_4	-4.107E+00	-5.558E+00	-6.938E+00	4.70

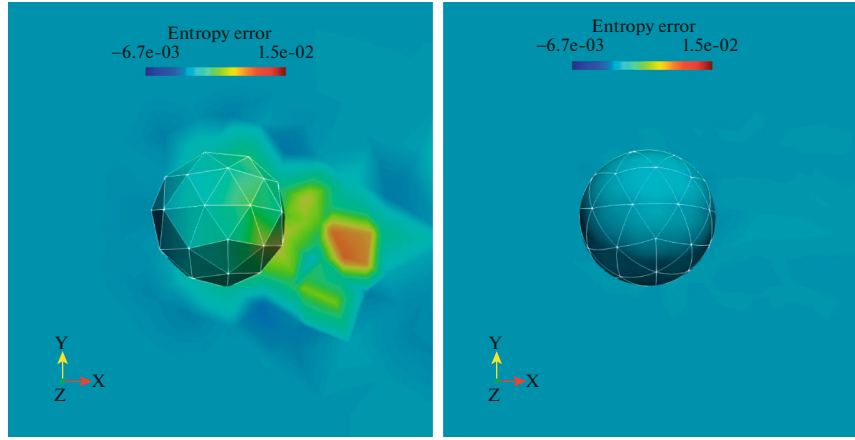


Fig. 4. Smooth flow past a sphere: comparison of entropy error with the curved mesh (left) and the linear mesh (right). The local entropy error $S_{\text{err}} := P/\rho^\gamma - P_0/\rho_0^\gamma$. The linear-element approximation of curved surface induces a large entropy wake.

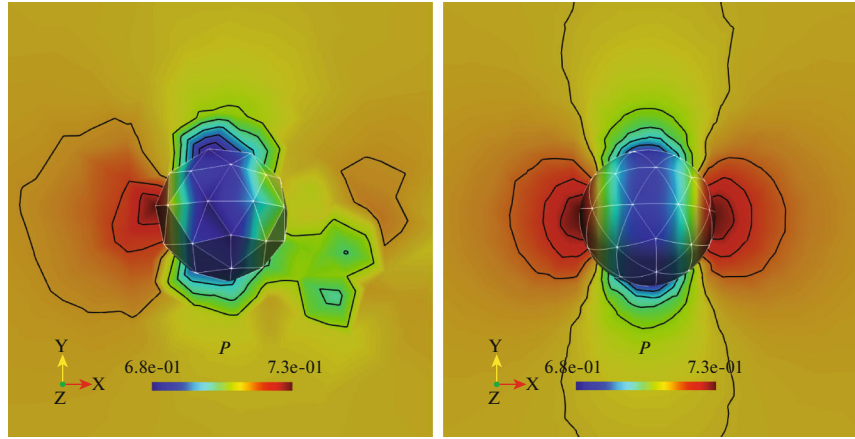


Fig. 5. Smooth flow past a sphere: comparison of pressure contours with the curved mesh (left) and the linear mesh (right). The result of the linear mesh gives an unphysical wake flow pattern.

can be obtained only if the curved mesh is employed, while the convergence of the linear mesh shows stagnation due to the production of unphysical wakes, as shown in Fig. 4. The error is measured quantitatively with the exact entropy value, which is defined as

$$E_{L_2(\Omega)}(s) := \sqrt{\frac{1}{|\Omega|} \int_{\Omega} \left(\frac{s}{s_0} \right)^2 dV} - 1, \quad (19)$$

where Ω is the cell element and $|\Omega|$ is the cell volume. $s := P/\rho^\gamma$ is the entropy, P the pressure and ρ the density. s_0 is the exact entropy value of the free stream. In Fig. 3 (left), it is observed that a steady-state convergence cannot be realized with the linear mesh. The entropy error with the linear mesh stagnates at $10^{-3.42}$, while the one with the curved mesh converged at $10^{-5.32}$, as shown in Fig. 3 (right). So the mesh curving does help in reducing the errors induced by the geometry approximations. In Fig. 4 (left), large, unphysical entropy errors with the linear mesh occur in the wake flow region due to the use of straight-sided surface approximations. Flow symmetry is lost as shown in the pressure contour of Fig. 5 (left). In contrast, using the recovered curved mesh delivers theoretically consistent results, in which the computed pressure contour is well symmetric, as shown in Fig. 5 (right). The local entropy error $S_{\text{err}} := P/\rho^\gamma - P_0/\rho_0^\gamma$ is much smaller than the linear one, see Fig. 4 (left) and Fig. 3 (right). This case shows that the developed

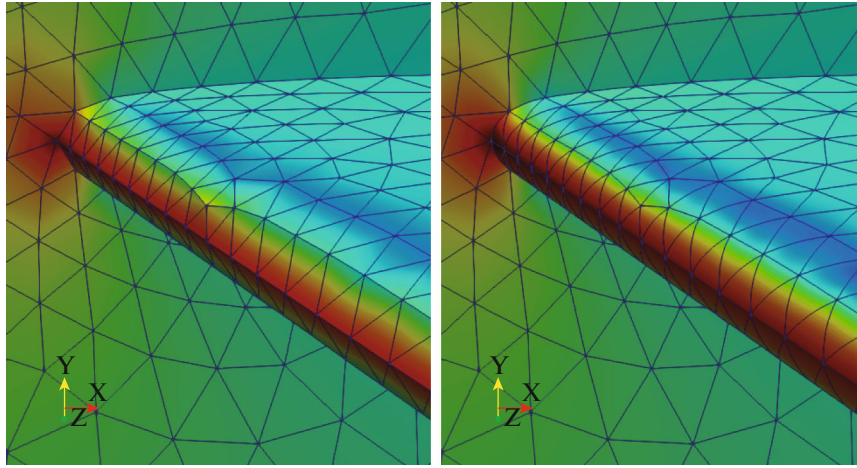


Fig. 6. Surface mesh of the ONERA M6 wing: P_0 solution, linear mesh (left); P_1 solution, curved mesh (right).

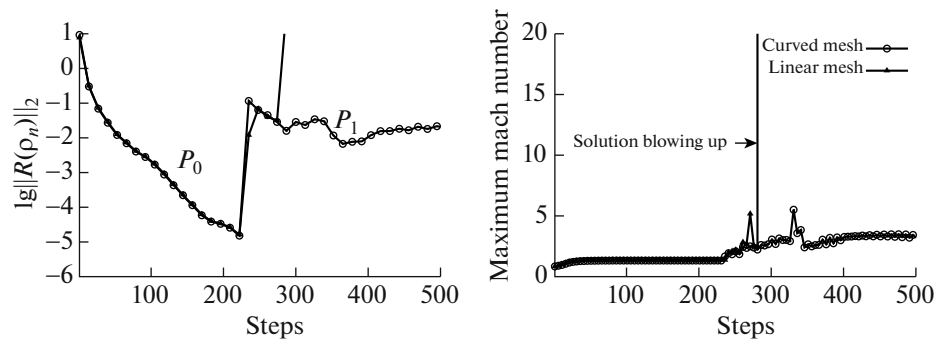


Fig. 7. ONERA M6 wing: residual convergence (left), maximum mach number (right). The computations are started with P_0 accuracy and then switched to P_1 accuracy for enhancing code robustness without using a limiter. The solution with linear elements quickly blows up after switching to the second-order accuracy. The steady convergence is monitored with the globally maximum number number of the flow field.

mesh curving method is indeed helpful in reducing entropy errors produced by the linear surface approximations.

4.3. ONERA M6 Wing

The ONERA M6 wing is computed using curved grids with the discontinuous Galerkin method. Comparisons to experiments are of particular interests for this case, in which code stabilities of using quadratic surface elements and traditional linear elements are also examined without using a discontinuity limiter. The flow condition uses the Mach number $M_\infty = 0.8395$ and angle-of-attack $\alpha = 3.06^\circ$ which produces the well-known λ -shape transonic shock waves. A coarse mesh consisting of 96222 tetrahedrons is used to distinguish the solution differences obtained from linear and curved meshes. The mesh is so coarse in the sense that the leading edge area is only discretized by two cells, as detailed in Fig. 6. From which, we see that the linear mesh is low resolution, and not surprising, resulting in a solution blowing up, see Fig. 7. In contrast, the solution obtained with curved elements robustly converges a quasi-steady solution where the maximal Mach number is approaching a constant. The computed results agree well with experimental data [10] in seven sectional locations η , as shown in Fig. 8.

4.4. DLR-F6 Wing/Body/Nacelle/Pylon

4.4.1. Mesh curving results. The mesh curving strategies are applied to a three-dimensional complex geometry: DLR-F6 wing/body/nacelle/pylon configuration. The input DLR-F6 surface mesh has

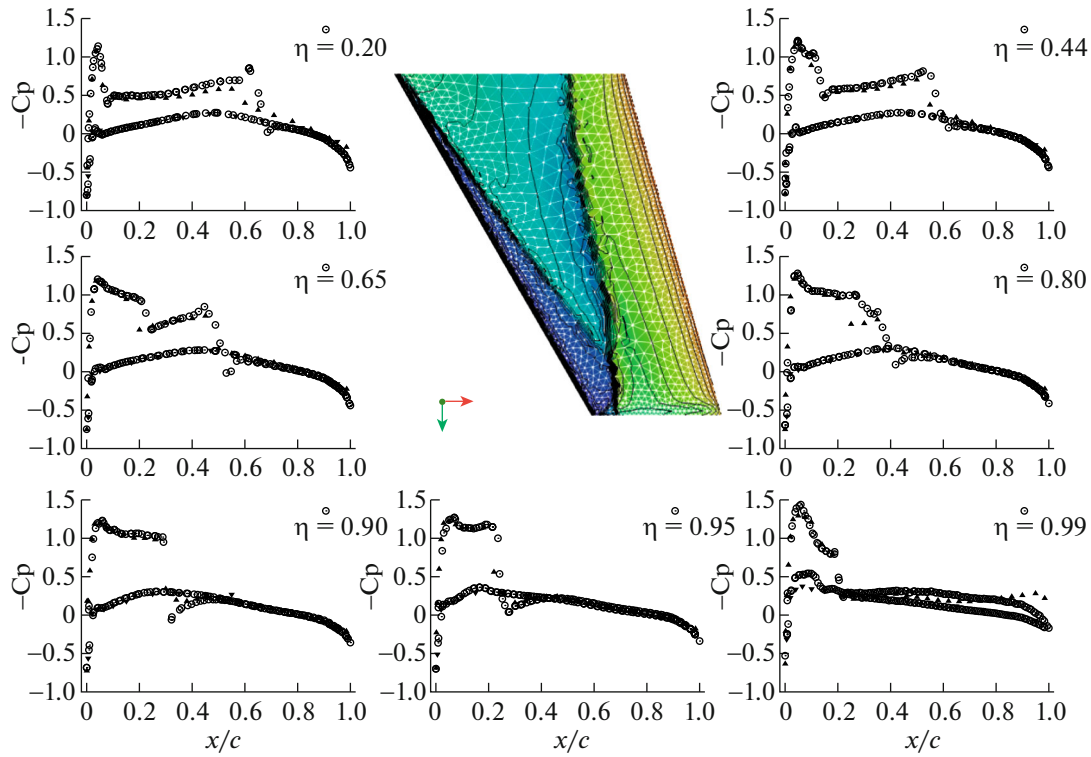


Fig. 8. ONERA M6 wing: pressure coefficient of P_1 solution with curved grids ($\odot$ 96222 cells) compared with the experimental data ($\blacktriangle$ upper surface, $\blacktriangledown$ lower surface).

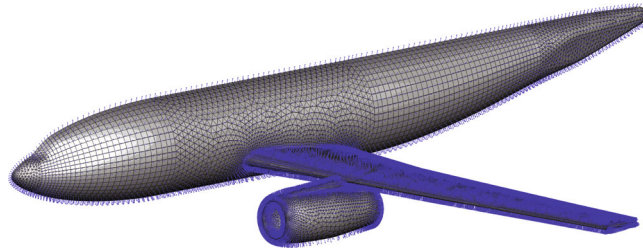


Fig. 9. DLR-F6 configuration: computed nodal normals; normal (blue), edge (black).

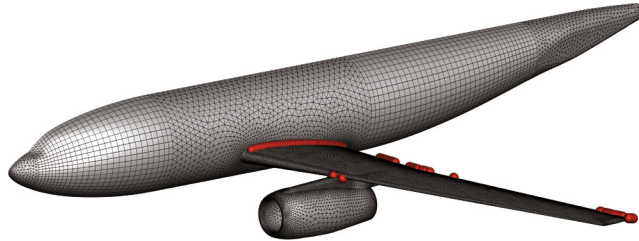


Fig. 10. DLR-F6 configuration: marking uncurved edges with red spheres.

34676 nodes, 54804 linear elements, 11150 linear edges. To obtain a curved mesh, firstly, nodal normals are approximated patch by patch according to Section 2.1.1. The computed normals at each node are shown in blue in Fig. 9. Although the current strategies cannot work perfectly for complex geometries, by applying the cubic Bézier surface reconstruction with $\alpha = 60^\circ$, 98.6% of the total 11150 edges can be con-

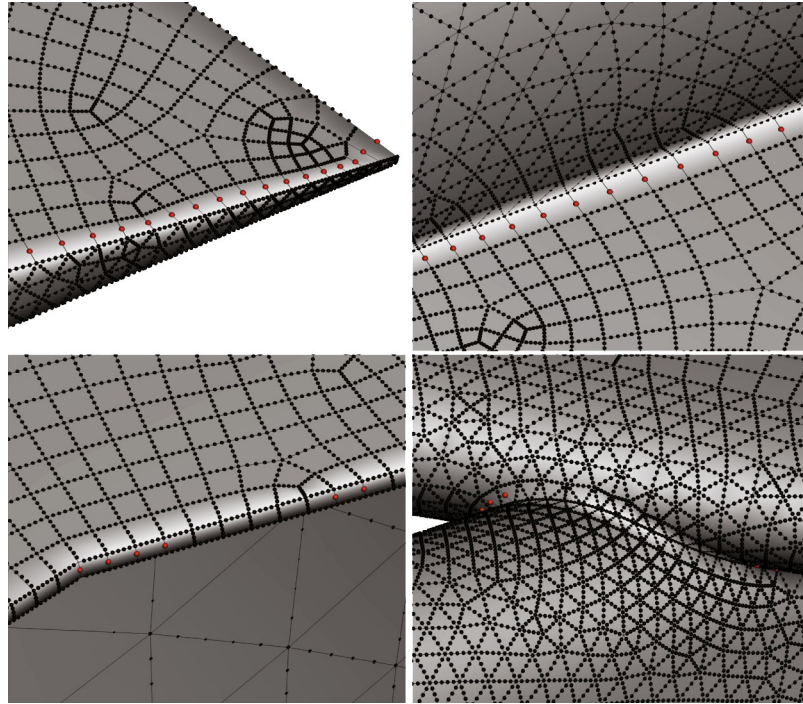


Fig. 11. DLR-F6 configuration: marking uncurved edges with the red sphere.

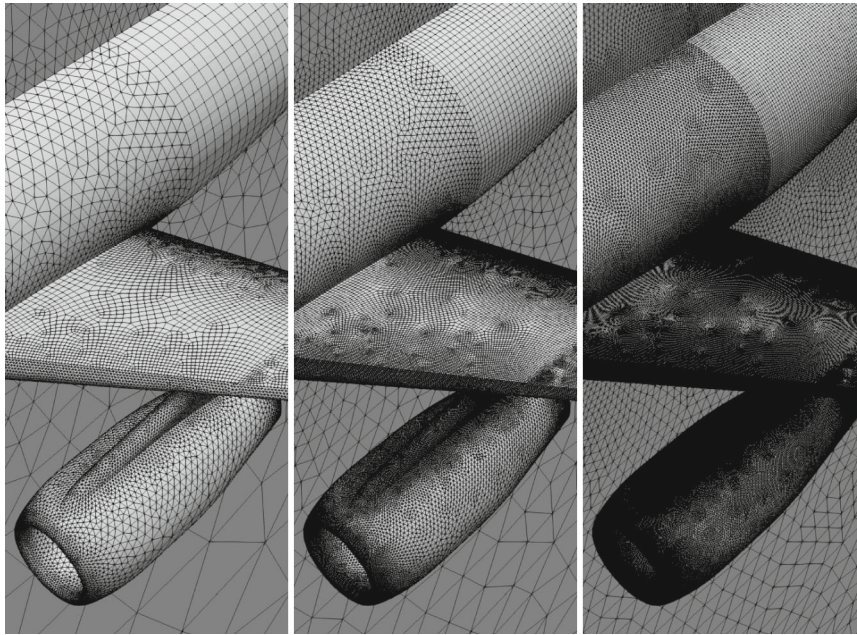


Fig. 12. DLR-F6 configuration: successive curved mesh refinements. Number of cells: 1345335 (left), 10791182 (middle), 86500468 $\approx$ 0.9 billion (right).

verted to curves and only 161 edges are kept linear. The uncurved, straight edges are marked with red spheres and are shown in Fig. 10. We can see that most of the high curvature regions are curved, especially those regions near the leading edge of the wing, which is important for generating pressure peaks. Zoom-in views of the uncurved regions are shown in Fig. 11, where the straight edges are marked with red spheres and the curved edges are denoted by multiple black nodes. Finally, the curved surface mesh is shown in the left one of Fig. 12, which is also used for curved mesh refinements in Section 3.3.

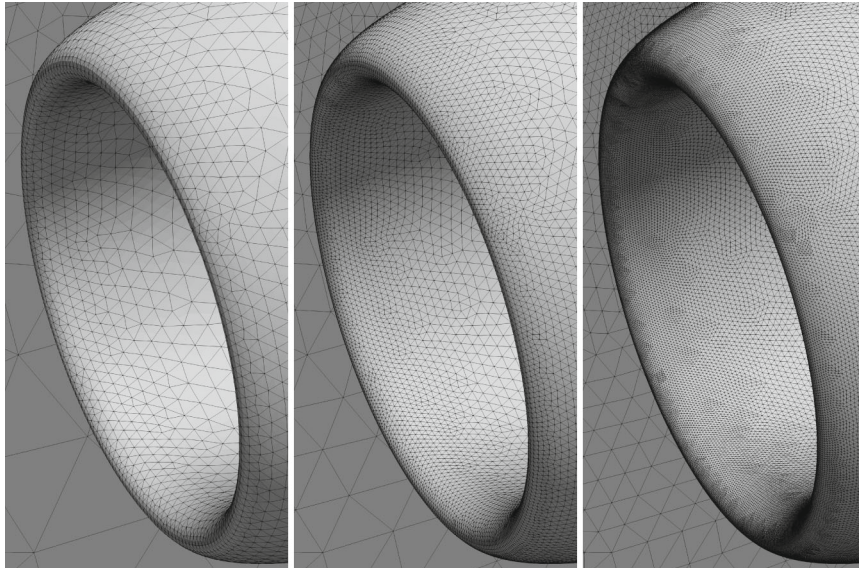


Fig. 13. DLR-F6 configuration: zoom-in views of the nacelle grids. The high-curvature surfaces are smoothly approximated with the curved mesh refinements.

4.4.2. Curved mesh refinement results. The mesh refinement method of Section 3.3 is applied to the DLR-F6 curved grids of Section 4.4.1. Two levels of successive mesh refinements are applied, resulting in curved cell numbers of 1345335, 10791182 and $86\,500\,468 \approx 0.9$ billions, as shown in Fig. 12 from left to right. Zoom-in views of the inlet of the nacelle are displayed in Fig. 13. It can be seen that the feature of the high-curvature surface of the inlet is smoothly captured for each level of curved mesh refinements, thanks to the reconstructed quadratic surfaces. Practical high-order computations over such a case are quite challenging, but the generated massively curved elements can be potentially useful for next-generation, high-order large eddy simulations of turbulent flows with massively curved elements.

5. CONCLUSIONS

Mesh curving and refinement algorithms are developed and investigated in the framework of high-order discontinuous Galerkin methods. The mesh curving algorithm uses the cubic Bézier surface/curves reconstruction to recover curvature from the provided linear mesh, thus can upgrade a linear mesh to a quadratic one. The effectiveness of the method is assessed in the DG methods for three-dimensional numerical solutions of smooth and discontinuous flows. Numerical results show that the use of mesh curving leads to more accurate results than using straight-sided meshes. From which unphysical entropy errors are significantly reduced for smooth flows, and excellent computational robustness is observed for all the test cases compared against using linear elements.

Additionally, the global curved mesh refinement method is further proposed based on the mesh curving algorithm. To our best knowledge, for the first time, up to 0.9 billions curved elements are successfully generated around the DLR-F6 wing/body/nacelle/pylon configuration. The resulted mesh refinement tool is potentially useful for future very large scale, high-order DG computations with massively curved elements of billion level. Finally, even if the mesh curving and refinement approaches are shown to be effective. State-of-art feature sharpening/feature detection and volume elements curving algorithms need to be further investigated for more complex geometries, which are of our future directions.

ACKNOWLEDGMENTS

This work is funded by the National Natural Science Foundation of China (NSFC) under the Grant U1530401. The author thanks Dr. Hang Si of WIAS, Germany for the discussion and collaboration in a broad sense of computational geometry. SJL would further like to thank the reviewers for their helpful comments.

REFERENCES

1. S.-J. Li, “A parallel discontinuous Galerkin method with physical orthogonal basis on curved elements,” *Procedia Eng.* **61**, 144–151 (2013).
2. S.-J. Li, Z. J. Wang, L. Ju, and L.-S. Luo, “Explicit large time stepping with a second-order exponential time integrator scheme for unsteady and steady flows,” *AIAA Paper* 2017-0753 (2017).
3. S.-J. Li, Z. J. Wang, L. Ju, and L.-S. Luo, “Fast time integration of Navier–Stokes equations with an exponential-integrator scheme,” *AIAA Paper* 2018-0369 (2018).
4. S.-J. Li, L.-S. Luo, Z. J. Wang, and L. Ju, “An exponential time-integrator scheme for steady and unsteady inviscid flows,” *J. Comput. Phys.* **365**, 206–225 (2018).
5. S.-J. Li, “Efficient p-multigrid method based on an exponential time discretization for compressible steady flows,” *arXiv:1807.0115*
6. S.-J. Li and L. Ju, “Exponential time-marching method for the unsteady Navier–Stokes equations,” *AIAA Paper* 2019-0907 (2019).
7. F. Bassi and S. Rebay, “High-order accurate discontinuous finite element solution of the 2D Euler equations,” *J. Comput. Phys.* **138** (2), 251–285 (1997).
8. L. Krivodonova and M. Berger, “High-order accurate implementation of solid wall boundary conditions in curved geometries,” *J. Comput. Phys.* **211** (2), 251–285 (2006).
9. E. Toro, *Riemann Solvers and Numerical Methods for Fluid Dynamics* (Springer, Berlin, 1999).
10. V. Schmitt and F. Charpin, “Pressure distributions on the ONERA-M6-wing at transonic Mach numbers,” *Experimental Data Base for Computer Program Assessment*, Report of the Fluid Dynamics Panel Working Group 04, AGARD AR 138, May 1979.
11. F. Bassi and S. Rebay, “A high-order accurate discontinuous finite element method for the numerical solution of the compressible Navier–Stokes equations,” *J. Comput. Phys.* **131** (2), 267–279 (1997).
12. F. Yamaguchi, *Curves and Surfaces in Computer Aided Geometric Design* (Springer, Heidelberg, 1988).
13. A. Vlachos, J. Peters, C. Boyd, et al., “Curved PN triangles,” in *Proceedings of the 2001 Symposium on Interactive 3D Graphics* (ACM, 2001).
14. O. Brodersen and A. Stürmer, “Drag prediction of engineairframe interference effects using unstructured Navier–Stokes calculations,” *AIAA paper* 2001-2414 (2001).
15. D. Shepard, “A two-dimensional interpolation function for irregularly-spaced data,” *Proceedings of the 1968 ACM National Conference* (1968), pp. 517–524.
16. F. Bassi, L. Botti, A. Colombo, D. A. Di Pietro, and P. Tesini, “On the flexibility of agglomeration based physical space discontinuous Galerkin discretizations,” *J. Comput. Phys.* **231** (1), 45–65 (2012).
17. K. D. Devine, E. G. Boman, R. T. Heaphy, et al., “Parallel hypergraph partitioning for scientific computing,” *Proceedings of 20th International Parallel and Distributed Processing Symposium (IPDPS’06)* (IEEE, 2006).
18. S. Zhang, “Successive subdivisions of tetrahedra and multigrid methods on tetrahedral meshes,” *Houston J. Math.* **21**, 541–556 (1995).
19. F. Hindenlang, T. Bolemann, and C.-D. Munz, “Mesh curving techniques for high order discontinuous Galerkin simulations,” *IDIHOM: Industrialization of High-Order Methods—a Top-Down Approach* (Springer, 2015), pp. 133–152.
20. H. Gao, Z. J. Wang, and Y. Liu, “Study of curved boundary representations for 2D high order Euler solvers,” *J. Sci. Comput.* **44**, 323–336 (2010).
21. J. Ims, Z. Duan, and Z. J. Wang, “meshCurve: An automated low-order to high-order mesh generator,” *AIAA Paper* 2015-2293 (2015).
22. X. Jiao and D. Wang, “Reconstructing high-order surfaces for meshing,” *Eng. Comput.* **28** (4), 361–373 (2012).